МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Лабораторна робота № 2
з дисципліни «Сучасні інженерні мережеві технології»

Тема: Автоматизація мережевих задач. Інвентаризація та перевірка контейнерної мережі засобами Python через Docker CLI та Docker Engine API

Виконав(ла): Плешу Андрій Валерійович, група ІО-36
Роботу прийняв: асистент кафедри ОТ Костянтин ГРІЩЕНКО

Київ — 2026

1. Мета і завдання лабораторної роботи

Мета: набути навичок автоматизованого збору мережевих даних засобами Python: обробка виводу Docker CLI через SSH (Paramiko) та робота зі структурованими даними Docker Engine API (Docker SDK for Python), перевірка доступності сервісів і формування звіту.

Завдання (високий рівень): утиліта netmon.py збирає інвентаризацію проєкту ЛР1 двома способами, нормалізує дані, порівнює їх, перевіряє публікацію портів і роботу застосунку через проксі, зберігає inventory.json та report.csv; у режимі monitor періодично опитує стан контейнерів і HTTP та пише monitoring.csv.

2. Результати виконання

2.1. Версії та середовище

Версії Python, бібліотек, Docker

Команда:

```
python --version; pip list; docker version; docker compose version
```

Результат:

```
$ .venv/bin/python --version; .venv/bin/python -m pip list 2>/dev/null | grep -iE '^(paramiko|d
  ocker|requests) '; docker version --format 'Docker Engine {{.Server.Version}}'; docker compos
  e version
Python 3.13.7
docker              7.2.0
paramiko            5.1.0
requests            2.34.2
Docker Engine 28.5.2
Docker Compose version v2.40.3-desktop.1
[exit code: 0]
```

Інтерпретація: Python 3.13, paramiko 5.1.0, docker (SDK) 7.2.0, requests 2.34.2; Docker Engine 28.5.2, Compose v2.40.3.

2.2. Структура проєкту, встановлення і запуск

- netmon.py — утиліта (режими inventory та monitor);
- requirements.txt — залежності з точними версіями;
- compose.yaml, ssh_host/ — контейнер lab2-sshhost, що імітує Linux VM з sshd та Docker CLI (вхід лише за ключем);
- setup.sh — генерація ключів, known_hosts, venv, запуск SSH-хоста;
- inventory.json, report.csv, monitoring.csv — результати.

```
pip install -r requirements.txt
./setup.sh
.venv/bin/python netmon.py inventory
.venv/bin/python netmon.py monitor --interval 10
```

Команди встановлення і запуску

Машиною керування є основна ОС (macOS), а роль Linux VM виконує контейнер lab2-sshhost (Alpine, sshd на 127.0.0.1:2222, доступ до Docker Engine через змонтований docker.sock). Паролі не використовуються; приватні ключі лежать у каталозі .state, який виключено з git. Ключ сервера перевіряється за локальним known_hosts (RejectPolicy, без автоматичного довіри).

2.3. Виконання пунктів завдання

Пункт 1. SSH-доступ до Docker і сервісів ЛР1

Команда:

```
ssh lab@127.0.0.1 -p 2222 'hostname; docker info; docker compose ls'
```

Результат:

```
$ ssh -i .state/client_key -p 2222 -o UserKnownHostsFile=.state/known_hosts -o StrictHostKeyChe
  cking=yes lab@127.0.0.1 'hostname; docker info --format "Docker Engine {{.ServerVersion}}"; d
```

```
  ocker compose ls'
lab-vm
Docker Engine 28.5.2
NAME                STATUS              CONFIG FILES
lab1                running(3)          /home/student/labs/LR1/docker-compose.yml
lab2-sshhost        running(1)          /home/student/labs/LR2/compose.yaml
[exit code: 0]
```

Інтерпретація: Підключення за ключем працює, користувач бачить Docker Engine, а проєкт lab1 запущений (3 контейнери).

Ручна перевірка застосунку

Команда:

```
curl http://127.0.0.1:8080/ (двічі)
```

Результат:

```
$ curl -s http://127.0.0.1:8080/; curl -s http://127.0.0.1:8080/
{"hits":3,"instance":"d5b4ae5d2b71"}
{"hits":4,"instance":"d5b4ae5d2b71"}
[exit code: 0]
```

Інтерпретація: Застосунок відповідає через проксі, лічильник у відповіді зростає — звернення до БД відбуваються.

Пункти 2-6. Інвентаризація, перевірки, HTTP, порівняння

Команда:

```
python netmon.py inventory
```

Результат:

```
$ .venv/bin/python netmon.py inventory
SSH: lab-vm | Docker: 28.5.2
NAME                STATUS              CONFIG FILES
lab1                running(3)          /home/student/labs/LR1/docker-compose.yml

[CLI] контейнерів: 3, [API] контейнерів: 3
  proxy  b0420c6c5dfe nginx:1.27-alpine  running  ['lab1-frontend=172.28.1.3'] ['0.0.0.0:8080->
  80/tcp', ':::8080->80/tcp']
  app    d5b4ae5d2b71 lab1-app:1.0       running  ['lab1-backend=172.28.2.3', 'lab1-frontend=17
  2.28.1.2'] []
  db     f1a5660c9a27 redis:7-alpine     running  ['lab1-backend=172.28.2.2'] []

Мережа -> контейнери:
  lab1-frontend: app, proxy
  lab1-backend: app, db

Перевірка публікації портів: відповідає вимогам
HTTP #1: status=200 time=8.8 ms hits=5
HTTP #2: status=200 time=4.4 ms hits=6
Лічильник зріс на 1 (proxy -> app -> db): True

Порівняння CLI та API: відмінностей немає

Збережено: inventory.json, report.csv
[exit code: 0]
```

Інтерпретація: Через SSH+CLI та через Docker SDK отримано по 3 контейнери; таблиця «мережа → контейнери» показує app у двох мережах, перевірка портів видає «відповідає вимогам» (порт публікує лише proxy), два HTTP-запити дали 200 і лічильник +1 (proxy → app → db), порівняння CLI та API не виявило відмінностей.

Пункт 7. inventory.json

Команда:

```
head inventory.json
```

Результат:

```
$ head -c 1500 inventory.json; echo; echo ...; .venv/bin/python -c "import json;d=json.load(ope
  n('inventory.json'));print({k:len(v['containers']) for k,v in d.items()})"
{
  "cli": {
    "host": "dev-host",
    "project": "lab1",
```

```
    "source": "cli",
    "collected_at": "2026-10-21T14:39:17+00:00",
    "containers": [
      {
        "service": "proxy",
        "id": "b0420c6c5dfe48963a415f30fe2776a85dcb1bb6cea593bd7e803522dec951c6",
        "image": "nginx:1.27-alpine",
        "state": "running",
        "started_at": "2026-10-21T14:37:36Z",
        "networks": [
          "lab1-frontend=172.28.1.3"
        ],
        "ports": [
          "0.0.0.0:8080->80/tcp",
          ":::8080->80/tcp"
        ]
      },
      {
        "service": "app",
        "id": "d5b4ae5d2b71a0b62263a4acd2e80a659b83e284850646de08b9a062ca60d92b",
        "image": "lab1-app:1.0",
        "state": "running",
        "started_at": "2026-10-21T14:37:30Z",
        "networks": [
... (скорочено, усього 60 рядків)
```

Інтерпретація: Файл містить два знімки зі спільною структурою (cli та api): хост, проєкт, час збору та для кожного контейнера сервіс, повний ID, образ, стан, час запуску, мережі, IP та порти.

report.csv

Команда:

```
cat report.csv
```

Результат:

```
$ cat report.csv
category,item,result,details
network_map,lab1-frontend,info,"app, proxy"
network_map,lab1-backend,info,"app, db"
ports_check,лише проксі публікує порт,PASS,відповідає вимогам
http,запит 1,PASS,status=200 time_ms=8.8 hits=5
http,запит 2,PASS,status=200 time_ms=4.4 hits=6
http,лічильник +1,PASS,"[5, 6]"
compare_cli_api,"склад, стани, мережі, порти",PASS,відмінностей немає
[exit code: 0]
```

Інтерпретація: Підсумкова таблиця містить результати всіх перевірок (мережі, порти, HTTP, порівняння CLI/API) у форматі CSV.

2.4. Обробка помилок

Недоступний вузол

Команда:

```
python netmon.py inventory --ssh-port 2299
```

Результат:

```
$ .venv/bin/python netmon.py inventory --ssh-port 2299
ПОМИЛКА: вузол недоступний або автентифікація не вдалася: [Errno None] Unable to connect to por
  t 2299 on 127.0.0.1
[exit code: 2]
```

Інтерпретація: Скрипт не падає з трасуванням, а повідомляє про недоступність вузла і завершується з кодом 2.

Невірний ключ сервера

Команда:

```
python netmon.py inventory --known-hosts bad_known_hosts
```

Результат:

```
$ echo '[127.0.0.1]:2222 ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIOMqqemGtaNeverSeenBeforeKeyAAAAAA
  AAAAAAAAAA' > .state/bad_known_hosts; .venv/bin/python netmon.py inventory --known-hosts .sta
```

```
  te/bad_known_hosts
ПОМИЛКА: вузол недоступний або автентифікація не вдалася: Host key for server '127.0.0.1' does
  not match: got 'AAAAC3NzaC1lZDI1NTE5AAAAIBvNdJ8btGdJ+aAmr6spOJ/AkWWB+lRw44zKAkm+Z6ht', expect
  ed 'AAAAC3NzaC1lZDI1NTE5AAAAIOMqqemGtaNeverSeenBeforeKeyAAAAAAAAAAAAAAAA'
[exit code: 2]
```

Інтерпретація: Підключення відхилено: ключ сервера не збігається з відомим (захист від MITM).

Помилка віддаленої команди

Команда:

```
run_ssh(ssh, 'docker inspect no-such-container')
```

Результат:

```
$ .venv/bin/python -c "
import netmon
a = netmon.parse.__globals__['argparse'].Namespace(ssh_host='127.0.0.1', ssh_port=2222, user='l
  ab', key=netmon.Path('.state/client_key'), known_hosts=netmon.Path('.state/known_hosts'))
ssh = netmon.ssh_connect(a)
try:
    netmon.run_ssh(ssh, 'docker inspect no-such-container')
except RuntimeError as e:
    print('RuntimeError:', e)
"
RuntimeError: команда завершилась з кодом 1: docker inspect no-such-container: Error: No such o
  bject: no-such-container
[exit code: 0]
```

Інтерпретація: Ненульовий код завершення віддаленої команди перетворюється на помилку з текстом stderr.

Порушення вимоги щодо портів

Команда:

```
docker run -p 9999:80 --label com.docker.compose.project=lab1 ...; python netmon.py inventory
```

Результат:

```
$ .venv/bin/python netmon.py inventory --report report_violation.csv --inventory inventory_viol
  ation.json
SSH: lab-vm | Docker: 28.5.2
NAME                STATUS              CONFIG FILES
lab1                running(3)          /home/student/labs/LR1/docker-compose.yml

[CLI] контейнерів: 4, [API] контейнерів: 4
  proxy  b0420c6c5dfe nginx:1.27-alpine  running  ['lab1-frontend=172.28.1.3'] ['0.0.0.0:8080->
  80/tcp', ':::8080->80/tcp']
  extra  d4291fe7fb7b nginx:1.27-alpine  running  ['bridge=172.17.0.2'] ['0.0.0.0:9999->80/tcp'
  , ':::9999->80/tcp']
  app    d5b4ae5d2b71 lab1-app:1.0       running  ['lab1-backend=172.28.2.3', 'lab1-frontend=17
  2.28.1.2'] []
  db     f1a5660c9a27 redis:7-alpine     running  ['lab1-backend=172.28.2.2'] []

Мережа -> контейнери:
  lab1-frontend: app, proxy
  bridge: extra
  lab1-backend: app, db

Перевірка публікації портів: порушення: extra (d4291fe7fb7b): 0.0.0.0:9999->80/tcp, :::9999->80
  /tcp
HTTP #1: status=200 time=9.7 ms hits=7
HTTP #2: status=200 time=11.8 ms hits=8
Лічильник зріс на 1 (proxy -> app -> db): True

Порівняння CLI та API: відмінностей немає

Збережено: inventory_violation.json, report_violation.csv
[exit code: 1]
```

Інтерпретація: Сторонній контейнер проєкту, що публікує порт 9999, виявлено: перевірка виводить назву контейнера і порти й завершується з кодом 1.

2.5. Додаткове завдання: періодичний моніторинг

Запуск моніторингу зі зупинкою та відновленням БД

Команда:

```
python netmon.py monitor --interval 3 --count 14 (паралельно docker stop db, через 12 с docker
  start db)
```

Результат:

```
$ .venv/bin/python netmon.py monitor --interval 3 --count 14
2026-10-21T17:39:21 app=running;db=running;proxy=running 200 4.5
2026-10-21T17:39:24 app=running;db=running;proxy=running 200 4.8
2026-10-21T17:39:28 app=running;db=running;proxy=running 200 14.8
2026-10-21T17:39:31 app=running;db=exited;proxy=running 503 15.7
2026-10-21T17:39:34 app=running;db=exited;proxy=running 503 8.7
2026-10-21T17:39:37 app=running;db=exited;proxy=running 503 9.8
2026-10-21T17:39:40 app=running;db=exited;proxy=running 503 10.1
2026-10-21T17:39:43 app=running;db=running;proxy=running 200 17.2
2026-10-21T17:39:46 app=running;db=running;proxy=running 200 9.2
2026-10-21T17:39:49 app=running;db=running;proxy=running 200 9.0
2026-10-21T17:39:52 app=running;db=running;proxy=running 200 7.1
2026-10-21T17:39:56 app=running;db=running;proxy=running 200 7.2
2026-10-21T17:39:59 app=running;db=running;proxy=running 200 7.4
2026-10-21T17:40:02 app=running;db=running;proxy=running 200 9.2
[exit code: 0]
```

Інтерпретація: Інтервал задається параметром --interval. Під час зупинки db стан показує db=exited, HTTP повертає 503, але моніторинг продовжується; після старту db HTTP знову 200.

Фрагмент monitoring.csv

Команда:

```
cat monitoring.csv
```

Результат:

```
$ cat monitoring.csv
timestamp,services,http_status,http_time_ms
2026-10-21T17:39:21,app=running;db=running;proxy=running,200,4.5
2026-10-21T17:39:24,app=running;db=running;proxy=running,200,4.8
2026-10-21T17:39:28,app=running;db=running;proxy=running,200,14.8
2026-10-21T17:39:31,app=running;db=exited;proxy=running,503,15.7
2026-10-21T17:39:34,app=running;db=exited;proxy=running,503,8.7
2026-10-21T17:39:37,app=running;db=exited;proxy=running,503,9.8
2026-10-21T17:39:40,app=running;db=exited;proxy=running,503,10.1
2026-10-21T17:39:43,app=running;db=running;proxy=running,200,17.2
2026-10-21T17:39:46,app=running;db=running;proxy=running,200,9.2
2026-10-21T17:39:49,app=running;db=running;proxy=running,200,9.0
2026-10-21T17:39:52,app=running;db=running;proxy=running,200,7.1
2026-10-21T17:39:56,app=running;db=running;proxy=running,200,7.2
2026-10-21T17:39:59,app=running;db=running;proxy=running,200,7.4
2026-10-21T17:40:02,app=running;db=running;proxy=running,200,9.2
[exit code: 0]
```

Інтерпретація: Три етапи: норма (db=running, 200), збій (db=exited, 503), відновлення (db=running, 200). Затримка між зупинкою БД і появою 503 не перевищує інтервалу опитування.

2.6. Ключові фрагменти коду

```
def collect_cli(ssh, project):
    containers = []
    ps = run_ssh(ssh, f"docker ps -a --no-trunc --filter label=com.docker.compose.project={project} "
                      "--format '{{.ID}} {{.Image}} {{.State}}'")
    for line in ps.splitlines():
        cid, image, state = line.split()
        service, started = run_ssh(
            ssh, "docker inspect --format "
                 "'{{index .Config.Labels \"com.docker.compose.service\"}} {{.State.StartedAt}}' " + cid).sp
        nets_raw = run_ssh(
            ssh, "docker inspect --format "
                 "'{{range $name, $network := .NetworkSettings.Networks}}"
                 "{{println $name $network.IPAddress}}{{end}}' " + cid)
        networks = {}
        for ln in nets_raw.splitlines():
            parts = ln.split()
```

```
            if parts:
                networks[parts[0]] = parts[1] if len(parts) > 1 else ""
        # docker port не повертає помилку для контейнера без портів
        ports_raw = run_ssh(ssh, f"docker port {cid}")
        ports = []
        for ln in ports_raw.splitlines():
            m = re.match(r"(\d+)/(\w+) -> (.+):(\d+)$", ln.strip())
            if m:
                ports.append((m.group(1), m.group(2), m.group(3).strip("[]"), m.group(4)))
        containers.append({"service": service, "id": cid, "image": image, "state": state,
                           "started_at": started, "networks": networks, "ports": ports})
    return containers
```

Збір через Docker CLI (Paramiko)

```
def collect_api(client, project):
    containers = []
    for c in client.containers.list(all=True, filters={"label": f"com.docker.compose.project={project}"}):
        c.reload()
        a = c.attrs
        networks = {n: d["IPAddress"] for n, d in a["NetworkSettings"]["Networks"].items()}
        ports = []
        for key, binds in (a["NetworkSettings"]["Ports"] or {}).items():
            port, proto = key.split("/")
            for b in binds or []:
                ports.append((port, proto, b["HostIp"], b["HostPort"]))
        containers.append({"service": c.labels["com.docker.compose.service"], "id": c.id,
                           "image": a["Config"]["Image"], "state": a["State"]["Status"],
                           "started_at": a["State"]["StartedAt"], "networks": networks, "ports": ports})
    return containers
```

Збір через Docker Engine API (Docker SDK)

```
def normalize(containers):
    out = []
    for c in containers:
        out.append({
            "service": c["service"],
            "id": c["id"],
            "image": c["image"],
            "state": c["state"].lower(),
            "started_at": norm_time(c["started_at"]),
            "networks": sorted(f"{n}={ip}" for n, ip in c["networks"].items()),
            "ports": sorted(f"{h}:{hp}->{p}/{proto}" for p, proto, h, hp in c["ports"]),
        })
    return sorted(out, key=lambda x: x["id"])
```

Нормалізація до спільної структури

```
def compare(cli, api):
    """Порівняння за повним ID: склад, стан, мережі, порти (без урахування порядку)."""
    diffs = []
    a = {c["id"]: c for c in cli["containers"]}
    b = {c["id"]: c for c in api["containers"]}
    for cid in sorted(set(a) - set(b)):
        diffs.append(f"контейнер {a[cid]['service']} ({cid[:12]}) є лише в CLI")
    for cid in sorted(set(b) - set(a)):
        diffs.append(f"контейнер {b[cid]['service']} ({cid[:12]}) є лише в API")
    for cid in sorted(set(a) & set(b)):
        for field in ("service", "image", "state", "started_at"):
            if a[cid][field] != b[cid][field]:
                diffs.append(f"{a[cid]['service']}: {field}: CLI={a[cid][field]} API={b[cid][field]}")
        for field in ("networks", "ports"):
            if set(a[cid][field]) != set(b[cid][field]):
                diffs.append(f"{a[cid]['service']}: {field}: CLI={a[cid][field]} API={b[cid][field]}")
    return diffs
```

Порівняння CLI та API за повним ID

```
class StrictSSHAdapter(SSHHTTPAdapter):
    """Адаптер Docker SDK із суворою перевіркою ключа сервера та ключем з аргументів."""
    settings = None

    def _create_paramiko_client(self, base_url):
        self.ssh_client = paramiko.SSHClient()
        s = StrictSSHAdapter.settings
        self.ssh_params = {"hostname": s.ssh_host, "port": s.ssh_port, "username": s.user,
                           "key_filename": str(s.key), "allow_agent": False, "look_for_keys": False, "timeou
        self.ssh_client.load_host_keys(str(s.known_hosts))
        self.ssh_client.set_missing_host_key_policy(paramiko.RejectPolicy())
```

```
def docker_api(args, ssh_client=None):
    """Docker Engine API поверх SSH (docker system dial-stdio)."""
    StrictSSHAdapter.settings = args
    # підміна адаптера, який створює APIClient
    docker.api.client.SSHHTTPAdapter = StrictSSHAdapter
    return docker.DockerClient(base_url=f"ssh://{args.user}@{args.ssh_host}:{args.ssh_port}",
                               use_ssh_client=False, timeout=10)
```

Docker SDK через SSH зі строгою перевіркою ключа

Нормалізація приводить стани до нижнього регістру, час запуску до ISO-8601 UTC без наносекунд, мережі та порти до відсортованих списків рядків. Тому порядок елементів не впливає на порівняння. Повний код — у файлі netmon.py.

3. Висновки

Створено Python-утиліту, що через SSH збирає інвентаризацію проєкту ЛР1 двома способами (текстовий вивід Docker CLI та структуровані дані Docker Engine API), зводить їх до спільної моделі й автоматично порівнює. CLI зручний для швидких одноразових команд, але потребує розбору тексту; API повертає структуровані дані й менш чутливий до формату виводу. Утиліта перевіряє мережеву конфігурацію, ланцюжок proxy → app → db та формує inventory.json і report.csv. Режим моніторингу показав, що збій БД видно за станом контейнера та HTTP-статусом 503, а відновлення фіксується наступним опитуванням.

Підпис виконавця: _____ Підпис викладача: _____